SecureDT - Complete Installation and Usage Guide

Version 1.0 | Django + MySQL + Groq AI + Genetic Algorithm

SECTION 1: PROJECT OVERVIEW

Problem: Efficient task scheduling across edge, MEC, and cloud resources with security risk optimization.
Objective: Intelligent, secure task scheduling optimizing latency, energy, privacy, and security risk.
Architecture: Django backend with REST API, MySQL database, GA scheduler, Digital Twin, AI risk engine (local RF + optional Groq LLM), attack simulation (safe software simulation), storage attack simulation, analytics, experiment frameworks.
Technology Stack: Python 3.13, Django 5.2, MySQL, PyMySQL, numpy, pandas, scikit-learn, reportlab, fpdf2, Flask, Git, HTML5/CSS3/JQuery, Chart.js.

SECTION 2: SOFTWARE REQUIREMENTS

Python: 3.10 or newer (64-bit recommended).
Django: 5.2 LTS (or 4.x+).
MySQL: 8.0+ with PyMySQL or mysql-connector-python.
Redis: Optional (for Celery fallback).
Browser: Chrome, Edge, or Firefox (latest).
Git: For version control and deployment.
Python Packages (from requirements.txt):
Flask>=2.3,<4.0
Flask-Cors>=4.0
numpy>=2.0
pandas>=2.0
scikit-learn>=1.3
reportlab>=4.0
fpdf2>=2.7

SECTION 3: PROJECT INSTALLATION

Step 1: Extract ZIP - Unzip the project archive to your desired location.
Step 2: Open terminal - Navigate to the project directory: cd path/to/securedt
Step 3: Create virtual environment - python -m venv .venv
Step 4: Activate environment - Windows: .venv\Scripts\activate | Linux/macOS: source .venv/bin/activate
Step 5: Install requirements - pip install -r requirements.txt
Step 6: Configure .env - cp .env.example .env Edit .env with your settings
Step 7: Create MySQL database - CREATE DATABASE securedt; CREATE USER 'securedt'@'%' IDENTIFIED BY 'password'; GRANT ALL PRIVILEGES ON securedt.* TO 'securedt'@'%';
Step 8: Configure MySQL - Update .env with DB settings.
Step 9: Run migrations - python manage.py makemigrations && python manage.py migrate
Step 10: Create superuser - python manage.py createsuperuser
Step 11: Run Django - python manage.py runserver Open http://127.0.0.1:8000 in browser

SECTION 4: MYSQL DATABASE

Database: securedt database with securedt user.
username: securedt
password: your_secure_password (set in .env)
host: localhost
port: 3306 (default MySQL port)
migration: python manage.py makemigrations && python manage.py migrate
tables: auth_user, plus application tables for tasks, attacks, storage, digital twin, analytics, AI prediction, Groq status.
Example (placeholders only):
DEFAULT: ENGINE=django.db.backends.sqlite3 NAME=BASE_DIR/db.sqlite3 (development)
MYSQL: ENGINE=django.db.backends.mysql NAME=securedt USER=securedt PASSWORD=your_password HOST=localhost PORT=3306

SECTION 5: USER ACCOUNT

Register: via /register with username, password, and email.
Login: via /login with username and password.
Logout: /logout ends the session.
Password reset: Django built-in flow via /password-reset.
Profile: accessible after login; displays role and preferences.

SECTION 6: DATASET SYSTEM

Synthetic datasets (seed-based reproducible) and optional live telemetry imports.
Download: python scripts/generate_dataset.py --tasks 10000 --attack 5000 --resource 3000
CSV upload validated for type/size/rows; normalized to UnifiedEvent schema.

SECTION 7: LARGE DATASET PROCESSING

Supports 100K to 10M+ rows using chunking, streaming, Parquet, DuckDB, MySQL metadata, background processing via Celery (optional), and pagination so browser never loads millions of rows.

SECTION 8: AI

Local AI: scikit-learn Random Forest - works without any API key.
Groq AI: Optional LLM insight layer via Groq Cloud API. Core functionality works offline; Groq adds natural-language explanations only.
Where API key goes: .env file ONLY - never in source, HTML, JS, CSS, or database.
Fallback mode: If Groq unavailable, app auto-falls back to local AI without crash.

SECTION 9: GENETIC ALGORITHM

Population: 40 chromosomes. Fitness weighted: latency (0.25), energy (0.15), privacy (0.20), security (0.30), deadline (0.10). Tournament selection (size 3), crossover (0.85), mutation (0.12), elitism (top 4). Generations: 50.

SECTION 11: DIGITAL TWIN

Virtual resource state mirror of edge/MEC/cloud nodes (CPU, memory, latency, trust, security-risk). Risk level updates dynamically. Attack events update risk scores and may quarantine nodes.

SECTION 13: ATTACK SIMULATION

SAFE SOFTWARE SIMULATION: No real attack occurs. No network traffic is sent.
Attacker: Simulated threat actor. Target: Selected MEC or Cloud resource. Risk increase: Security risk score based on attack severity. Detection: Security monitor raises alert. Digital Twin update: Resource state updated. AI analysis: Local AI provides risk explanation. GA reoptimization: Scheduler re-runs with updated risk weights. Rerouting: Tasks rerouted to healthier resources. Recovery: Mitigation stops attack, risk decreases, node recovered.

SECTION 14: STORAGE ATTACK

Virtual storages STORAGE-A (1000GB encrypted), STORAGE-B (2000GB encrypted), STORAGE-C (500GB unencrypted). Simulated attacker, integrity risk scores, detection, protection, isolation (affected storage quarantined; others unaffected), safe storage (simulation only), rerouting, recovery.

SECTION 15: SECURITY

Authentication: Django built-in auth, session-based login/register/logout.
Authorization: Role-based access control; superuser vs. regular user.
CSRF: Protected on all state-changing API requests via token.
XSS: Content Security Policy and output encoding prevent XSS.
SQL injection defense: Parameterized queries; Django ORM prevents SQLi.
File upload: CSV only, 8MB max, 100K row max, re-encoded on upload.
Rate limiting: Per-IP limiter (default 60 requests/minute).
Session security: Signed with SECRET_KEY; HttpOnly/SameSite cookies.
Security headers: CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, HSTS.

Audit logs: All events recorded with category, severity, and export capability.

SECTION 16: EXPERIMENTS

Baseline: random, greedy, privacy-aware schedulers. Proposed: Security-aware Genetic Algorithm. Dataset size: 100 to 1M+ tasks. AI: Risk model predictions integrated into fitness function. Metrics: Success rate, latency, energy, privacy exposure, security risk. Results: Comparison charts in /analytics and export to CSV/JSON/PDF.

SECTION 17: PERFORMANCE

Latency: Typically < 200ms for API responses; GA optimization ~ 5-30 seconds. Processing time: 1M rows ~ 30-60 seconds. Energy: Optimized scheduling reduces energy vs. random placement. Privacy: No personal data transmitted. Security: All secrets in environment variables. Scalability: Designed for 10K+ tasks; large datasets via chunking/streaming.

SECTION 18: REPORT GENERATION

CSV: Experiment results, analytics data, task summaries. JSON: API responses, system state exports. PDF: Generated experiment reports with charts and metrics. Charts: Chart.js-based visualizations. Experiment reports: /api/export/experiments generates full report.

SECTION 19: TROUBLESHOOTING

MySQL not connecting: Verify .env DB settings, MySQL service running, port 3306 open.
Groq key missing: Set GROQ_API_KEY in .env; app falls back to local AI.
Groq unavailable: Set AI_PROVIDER=localfeatures in .env.
Model missing: Run scripts/train_ai.py to generate models/risk_model.pkl.
Dataset missing: Generate with scripts/generate_dataset.py or upload CSV.
Celery unavailable: System works in synchronous mode; optional Celery/Redis for async.
Redis unavailable: Fallback to database queue or synchronous processing.
PDF generation error: Ensure reportlab is installed; check write permissions.

SECTION 20: HOW TO START

1. Extract ZIP - Unzip the project archive to your desired location.
2. Install Python - Python 3.13+ from python.org
3. Install MySQL - MySQL 8.0+ from mysql.com
4. Create database - CREATE DATABASE securedt; CREATE USER 'securedt'@'%' IDENTIFIED BY 'password';
5. Create .env - cp .env.example .env Edit with your DB, Groq, and app settings
6. Add GROQ_API_KEY - GROQ_API_KEY=YOUR_KEY_HERE (optional; local AI works without it)
7. Install dependencies - pip install -r requirements.txt
8. Run migrations - python manage.py makemigrations && python manage.py migrate
9. Create superuser - python manage.py createsuperuser
10. Start server - python manage.py runserver
11. Open browser - http://127.0.0.1:8000
12. Register - Create a new account or use: username=admin, password=admin123
13. Login - Access the dashboard and all features
14. Run Demo - Go to /experiments to compare schedulers, /attack to simulate attacks, /digital-twin to monitor resources